

1. 项目在做什么：

"从图片自动生成可交互3D场景"的智能系统

用户上传几张图片 → 系统自动做3D重建 → 自动生成Gaussian场景 → 自动导入Unity → 自动组织场景 -
> 最终用户可以在Unity中自由浏览交互

也就是：3DGS + Agent + Unity 的完整自动化pipeline

2. 核心技术

1. 3DGS

- 负责**图片恢复三维场景**
- 实时渲染
- 提供高质量视觉效果

2. Agent

- 自动调用各种工具
- 自动执行Pipeline
- 自动组织 workflow
- 原因：3DGS并非一键完成,中间需要**数据预处理, COLMAP, 点云生成, Gaussian训练, 格式转换, Unity导入, 场景组织** → Agent用于自动串联这些步骤

3. 为什么3DGS重要

过去方法 NeRF :用MLP隐式表示场景,渲染时需要大量神经网络计算

3DGS核心思想:**用大量 3D Gaussian 椭球表示场景**,每个高斯椭球包含位置,颜色,透明度,方向,尺寸 →然后把这些高斯椭球投影到屏幕上进行渲染

- **渲染速度极快**
- **支持实时显示**
- **更适合 Unity/VR**

4. 3DGS核心技术路线

step 1:输入RGB图片

step 2:SfM(structure from Motion) ⇒ COLMAP:**从多张图片中恢复相机位姿+稀疏点云**,也就是每张图片相机的位置和场景的大致结构

- **找特征点**:例如:墙角,桌角,纹理
- **图片匹配**:例如,发现"这两张图片都有这个桌角"
- **计算相机位置**:推算拍摄时,相机在哪
- **恢复稀疏点云**:得到场景大致结构
 - **点云**:一堆空间点,每个点只有x,y,z,只能说明这个地方有东西,但没有表面,没有材质,没有真实感

step 3:初始化 Gaussian 点:根据点云(在每个点附近生成Gaussian),创建大量 Gaussian,每个Gaussian可以理解成 **一个带颜色的3D 小椭球**,代表连续的概率分布

- **为什么是椭球** → 因为椭球可以表达方向,例如 **墙面**,Gaussian会被拉长,形成**扁平结构**,**树叶**:可能很薄,**毛发**:可能很细长.因此比球体灵活很多

step 4:投影到 2D 图像:把3D Gaussian根据相机参数,投影到屏幕,类似光栅化(区别在于三角形光栅化是看三角形覆盖哪些像素,而Gaussian是看椭圆斑点覆盖哪些像素)

- **Gaussina Rasterization** → 把Gaussian投影成2D 椭圆,然后混合颜色(**把3D 空间里的高斯椭球,投影到2D屏幕,并计算他们对每个像素的颜色贡献**)

step 5:渲染 + 反向传播优化:系统对比渲染结果和原始图片的误差,通过梯度下降优化 Gaussian位置,大小,颜色,透明度

step 6:Density Control(密度控制) → 自动增加/删除/分裂 Gaussian ,让场景细节越来越好

- **分裂Gaussian**:某个区域细节不足,就一个变多个
- **删除Gaussian**:如果某个Gaussian没有贡献,就删掉
- **调整密度**:复杂区域更多Gaussian,平坦区域更少Gaussian

5.Agent

调用图像预处理工具 → 调用COLMAP生成位姿 → 调用 Gaussian Splatting 训练 → 调用格式转换工具 - > 调用Unity导入脚本 → 自动创建场景,添加交互,设置材质,添加控制器

3DGS详解

1. 根据输入图片,恢复出3D世界

- 方法一:**Mesh(网络)**,即用三角形和顶点拼成模型,难自动生成,细节容易丢失
- 方法二:**Voxel(体素)**,把空间切成立方体,导致内存爆炸,分辨率低
- 方法三:**NeRF(神经辐射场)**,用神经网络表示整个世界,输入:相机位置+方向,输出:颜色,密度.**太慢**,每个像素都要跑神经网络

2. **为什么Gaussian适合渲染:**

- **平滑**:不会像voxel一样有锯齿
- **可以透明混合**:多个Gausssian叠加起来非常自然,形成的物体效果好

3. **为什么Gaussian快:**

- **不需要跑神经网络**,本质更接近**图形学渲染**,GPU非常擅长,可以做到实时,高帧率

Mip-Splatting(抗锯齿优化)

普通3DGS,当相机远离场景时,会出现 : 闪烁 / 摩尔纹 / 高频噪声 / 细节抖动,原因是Gaussian太小,投影到屏幕时,采样不稳定.

Mip-Splatting → **不同距离使用不同尺度的Gaussian**

- **近距离** → 使用小Gaussian保留细节
- **远距离** → 使用大Gaussian减少闪烁,画面更稳定

